Who Wrote It? Identifying LLMs from Their Responses

CMPSC 448 Midterm Project Report

Member: Joshua Wong (Individual Project)

Code: GitHub repository URL

Summary

We study whether the large language model (LLM) behind a piece of text can be identified from the text alone. Using 50,980 responses from 17 LLMs and chatbot products in the public TXD-22 dataset, we train a convolutional network (TextCNN) and a recurrent network (bidirectional LSTM) together with a TF-IDF + logistic-regression baseline. On test questions never seen during training, the BiLSTM reaches 0.636 macro-F1 and the TextCNN 0.632, about eleven times chance (0.059); the TF-IDF baseline is stronger at 0.723 (RQ1). The user's prompt carries no information about the source in this dataset: input-only classifiers are at chance, and adding the prompt to the response slightly lowers performance (RQ2). For extra credit, RQ4 shows that the fingerprint is mostly linguistic: response length, word choice, connective phrases such as *additionally* or *in summary*, and punctuation habits, while visual formatting carries almost no signal in this dataset.

1. Problem Definition and Dataset Curation

Task. Each example is a triple (LLM_name, LLM_input, LLM_output). RQ1 is a 17-way text classification problem: given LLM_output, predict LLM_name. RQ2 compares three inputs: the prompt only, the output only, and both. RQ4 asks which characteristics of the output make the LLMs distinguishable. We report accuracy and **macro-F1** (the unweighted mean of per-class F1), so every LLM counts equally.

Source. We use TXD-22: A Large-Scale Benchmark Dataset (Kaggle, mdsadiqiqbal/txd-22-a-large-scale-benchmark-dataset), a public CSV with the columns *Question* (prompt), *Answer* (response) and *Source*. It contains 66,000 rows: 3,000 answers from each of 22 sources to 3,076 distinct questions. The questions span many domains, for example computer science, physics (Lenz's law, Maxwell's equations), philosophy (Machiavelli, Aristotle), statistics and biology. The notebooks download the file with *kagglehub*; the raw data is not redistributed in our repository.

Curation steps.

- **Removing non-LLM labels.** Inspecting the 22 sources showed that five are not LLMs: *Human written*, *Human-written and AI-refined*, *AI-generated and AI-refined*, *AI-generated and AI-refined and Human-written* and *AI-generated and Human-written*. Because our research questions are about identifying LLMs, we drop them with a single configurable regular expression. This leaves 17 sources.
- **Duplicates.** We drop exact duplicate (answer, source) pairs and every answer that appears under more than one source, since its label is ambiguous (48 rows in total).
- **Encoding.** The CSV contains some invalid UTF-8 bytes. We read it with replacement characters (U+FFFD) instead of failing; Section 4.2 shows this matters.
- **Balance.** We cap each class at 3,000 answers and require at least 500. After cleaning every class keeps close to 3,000 answers (50,980 in total), so the classes are almost perfectly balanced.

The 17 sources are BLACKBOXAI, ChatGPT, Claude, Copilot, DeepAI, DeepSeek, Gemini, Gemma, Grok, Llama, Mistral AI, Nova, Perplexity, Pi, Poe, Qwen and Z.ai, which covers many more than the required three LLM families (OpenAI, Anthropic, Google, Meta, xAI, Alibaba, DeepSeek, Mistral, Amazon, Inflection, Zhipu). Note that several sources are *products* rather than model families: Copilot and Perplexity can run OpenAI models, and Poe, BLACKBOXAI and DeepAI give access to models from other companies. This turns out to explain the main errors in RQ1.

Split by question. 93.9% of questions were answered by all 17 sources. A random split would therefore put the same question in training and test, letting a model exploit topic overlap. We use scikit-learn's *StratifiedGroupKFold* with the question as the group, which keeps all answers to a question in one split while keeping the classes balanced. The same split (seed 42) is used for every experiment:

Split	Answers	Share	Distinct questions	Answers per LLM
Train	36,369	71.3%	2,197	approx. 2,140
Validation	7,296	14.3%	439	approx. 430
Test	7,315	14.3%	440	427 to 433

Table 1. Question-grouped data split. No question appears in more than one split.

Response length already differs strongly between sources: Pi averages 85 words per answer and Gemma 142, while Claude averages 295, Copilot 302, DeepAI 314 and BLACKBOXAI 344. Prompts, by contrast, are short (6.4 words on average) and the same for every source.

2. CNN and RNN Implementation and Training

2.1 Text preprocessing

We use a simple regular-expression tokenizer that lower-cases words and keeps every punctuation mark as its own token, plus a special <nl> token for line breaks, because punctuation and formatting are plausible stylistic fingerprints. The vocabulary is built from the **training split only** (tokens seen at least twice, at most 50,000), giving 34,517 tokens; unknown tokens map to <unk>. Each answer is truncated to its first 400 tokens and padded. The median answer has 283 tokens and 14.3% of answers are truncated.

2.2 Architectures

	TextCNN (Kim, 2014)	BiLSTM
Embedding	34,517 x 128, trained from scratch	34,517 x 128, trained from scratch
Encoder	3 parallel Conv1d layers, kernel sizes 3/4/5, 128 filters each, ReLU	1-layer bidirectional LSTM, hidden size 128 per direction, packed sequences so padding is skipped
Pooling	max-over-time per filter, concatenated (384-d)	final forward and backward hidden states, concatenated (256-d)
Regularization	dropout 0.5 before the classifier	dropout 0.5 on embeddings and before the classifier
Classifier	linear 384 to 17	linear 256 to 17
Parameters	4,621,713	4,686,737

Table 2. Model details. Most parameters are in the embedding layer (4.4M).

The CNN's kernels act as n-gram detectors (three- to five-token phrases such as *in summary ,*), and max-pooling records whether each pattern occurs anywhere in the answer. The BiLSTM reads the whole sequence in both directions, so in principle it can also model order and long-range structure. Using *pack_padded_sequence* ensures the LSTM's final state comes from the last real token, not from padding.

2.3 Training procedure

Both networks share one training loop: cross-entropy loss, AdamW (learning rate 1e-3, weight decay 1e-4), batch size 64, gradient-norm clipping at 1.0, and at most 10 epochs. After every epoch we compute validation macro-F1; training stops when it has not improved for 3 epochs, and the weights from the best epoch are restored. All random seeds are fixed (Python, NumPy, PyTorch) and every hyperparameter lives in a single configuration dataclass. Training ran on a Colab T4 GPU: the TextCNN took 82 s (best epoch 5, stopped after epoch 8) and the BiLSTM 223 s (best epoch 8 of 10).

Baselines. A majority-class classifier gives the chance floor. TF-IDF + logistic regression (word unigrams and bigrams, up to 100,000 features, sublinear term frequency) is a strong classical reference.

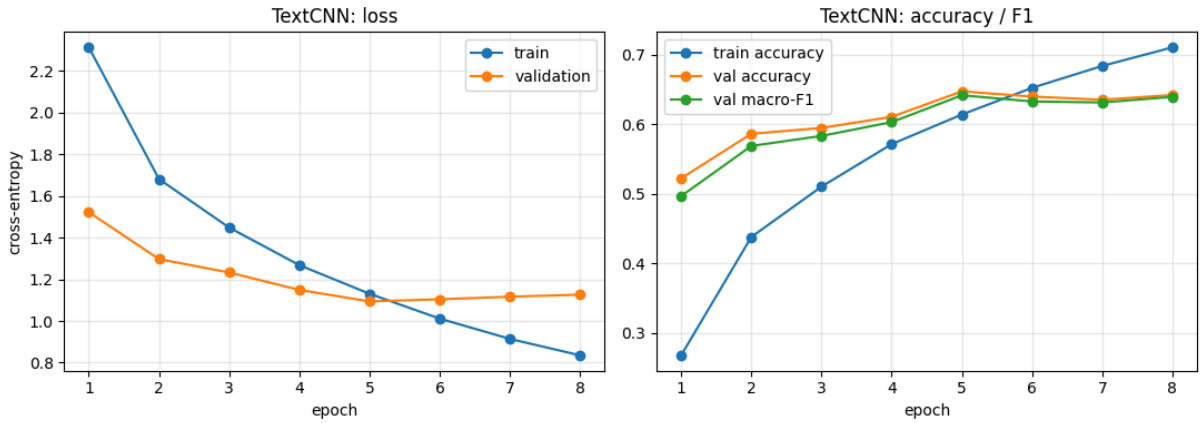


Figure 1. TextCNN training curves. Validation loss is lowest at epoch 5; afterwards the model keeps fitting the training set while validation accuracy flattens (overfitting).

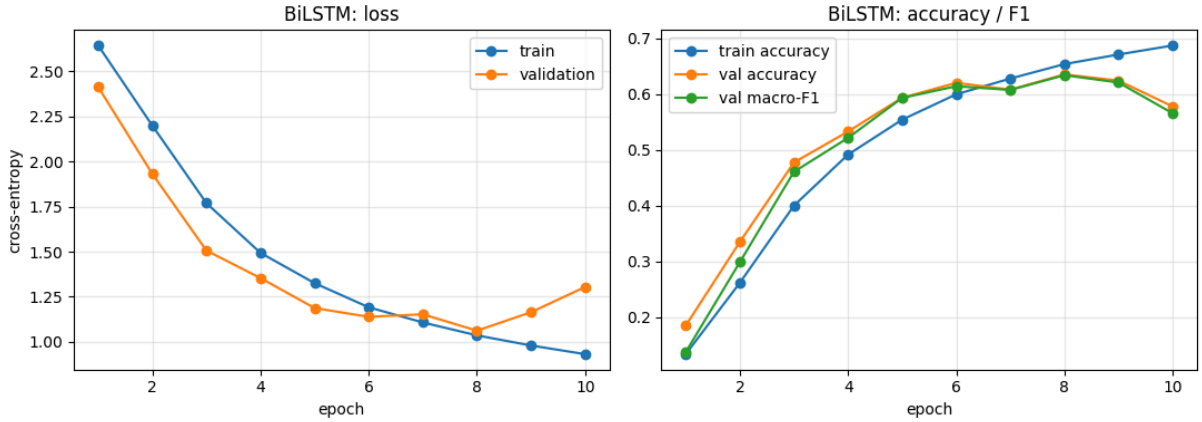


Figure 2. BiLSTM training curves. It learns more slowly than the CNN, peaks at epoch 8, and validation loss rises afterwards. Early stopping restores the epoch-8 weights.

3. Results

3.1 RQ1: Can we identify which LLM generated a response?

Model (input: response only)	Test accuracy	Test macro-F1
Majority class (chance floor)	0.058	0.007
TF-IDF + Logistic Regression	0.729	0.723
TextCNN	**0.638**	**0.632**
BiLSTM	**0.636**	**0.636**

Table 3. RQ1 results on 7,315 test answers to 440 unseen questions (17 classes, chance = 0.059).

Yes: LLMs can be identified from their responses well above chance. Both required neural models reach about 0.63 to 0.64 macro-F1, roughly eleven times the chance level, on questions they never saw. The two architectures perform almost identically overall, but differ per class (Figure 3): the BiLSTM is better on Pi, Gemma and Z.ai, while the TextCNN is better on BLACKBOXAI and DeepAI.

The TF-IDF baseline outperforms both networks by about 0.09 macro-F1. We think there are three reasons. (1) With only about 2,100 training examples per class, the networks must learn 4.4M embedding parameters from scratch and start to overfit after 5 to 8 epochs (Figures 1 and 2). (2) TF-IDF sees the entire answer, while the networks only see the first 400 tokens. (3) The fingerprint is largely *which words and phrases appear how often* (Section 4.2), which is exactly what a bag-of-n-grams model captures.

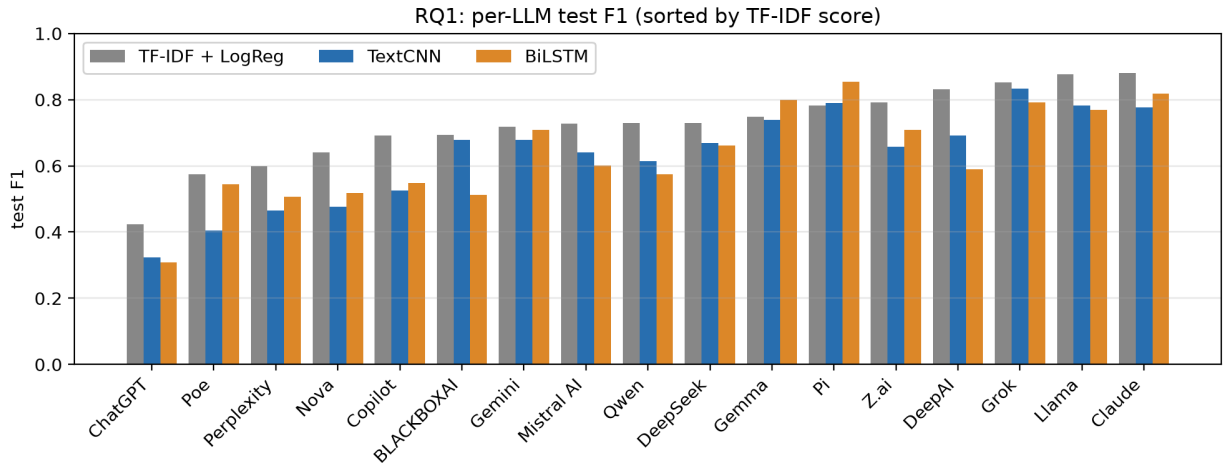


Figure 3. Per-LLM test F1 for the three models.

Easy and hard sources. Claude, Llama, Grok and Pi are the most identifiable (F1 0.78 to 0.88 for the best model). ChatGPT is consistently the hardest (F1 0.31 to 0.42), followed by Poe, Perplexity and Nova. The TF-IDF confusion matrix (Figure 4) shows why: ChatGPT answers are mistaken for Perplexity (17%), Copilot (15%) and DeepAI (10%), and 34% of Poe answers are predicted as BLACKBOXAI. These are exactly the sources that are platforms reselling or wrapping other companies' models, so they plausibly share an underlying model with ChatGPT or with each other. Smaller confusions occur within related families, for example Gemma and Pi (10%) and Mistral AI and Nova (12%).

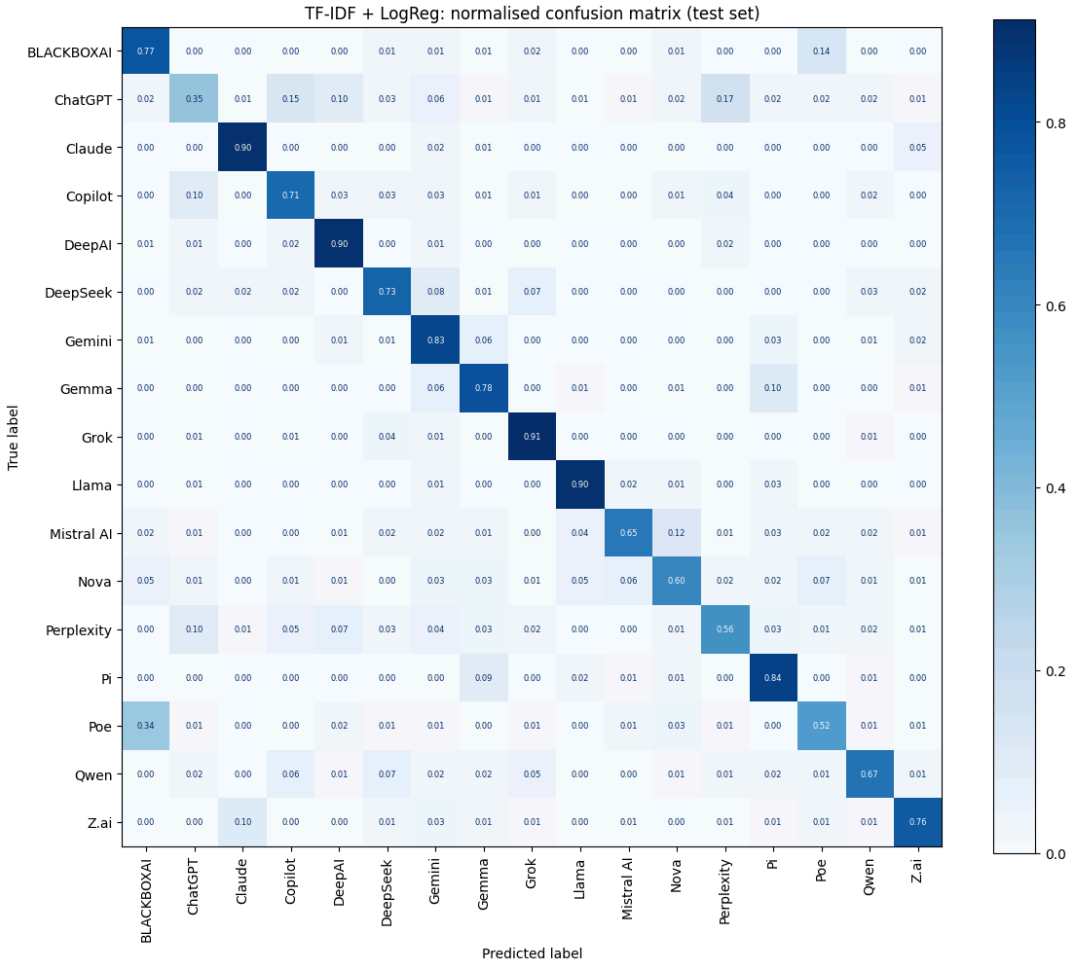


Figure 4. Normalized confusion matrix of TF-IDF + Logistic Regression on the test set (rows: true source).

3.2 RQ2: Does the user's prompt help identify the LLM?

Model	Input only	Output only	Input + Output	Change from adding input
Majority class	0.007	0.007	0.007	-
TF-IDF + LogReg	0.038	0.723	0.712	-0.011
TextCNN	0.042	0.640	0.634	-0.006
BiLSTM	0.028	0.645	0.628	-0.018

Table 4. RQ2 test macro-F1 (same question-grouped split and models as RQ1). Accuracies show the same pattern: 0.058 to 0.060 for input only.

For Input + Output, the TF-IDF model uses separate vectorizers for prompt and response, and the neural models read *prompt tokens <sep> response tokens*, with the prompt capped at 64 tokens so that the response part is identical to the Output-only setting.

- **(i) Can the LLM be predicted from the prompt alone?** No. All input-only models are at chance level (accuracy 0.058 to 0.060 versus 0.059 chance).
- **(ii) Does including the input improve classification?** No. Adding the prompt lowers macro-F1 slightly for all three models (-0.006 to -0.018). The prompt adds features that are uninformative about the source, which the models partly fit as noise.
- **(iii) Why would input-only classification work, and why does it not here?** It could only work if different LLMs received different prompts, for example because each was queried on its own topics or with its own template. That is a property of how a dataset was collected, not an LLM fingerprint. In TXD-22, 93.9% of questions were sent to all 17 sources and prompt lengths are the same for every source

(6.4 words), so the prompt cannot carry information about the source. The highest-weighted prompt words per source are topic words such as *Lenz's law* or *Machiavelli* that are shared by many sources, consistent with pure noise.

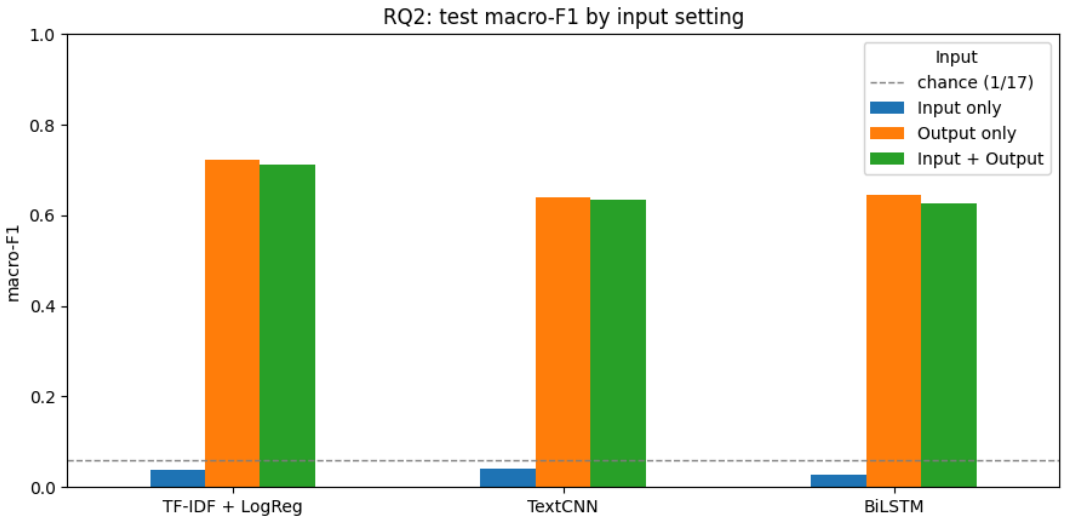


Figure 5. RQ2 test macro-F1 by input setting. Dashed line: chance.

Leakage check. We repeated input-only classification on a *random* split, where 99.9% of test prompts also appear in training. A lookup table that predicts the source that answered the same prompt in training scores **0.000** accuracy, and TF-IDF scores 0.0003, both far *below* chance. The reason is instructive: each source answered each question at most once, so the training copies of a test question always come from the *other* 16 sources, and memorization predicts exactly the wrong labels. This confirms that prompts carry no positive signal here, and it shows how much an evaluation result can depend on the split design.

Consistency check. The Output-only setting in RQ2 repeats RQ1 on the same split. TF-IDF reproduces RQ1 exactly (0.723). The neural models differ slightly (TextCNN 0.640 versus 0.632, BiLSTM 0.645 versus 0.636) because GPU kernels are not bit-for-bit deterministic even with fixed seeds. The run-to-run variation is therefore about 0.01, and differences smaller than that should not be over-interpreted.

4. In-depth Analysis (RQ4, extra credit): What Distinguishes the LLMs?

4.1 Interpretable style features

For every response we computed 28 interpretable features in six groups: **Length** (words, characters, sentences, words per sentence); **Formatting** (lines, blank lines, Markdown headings, bullets, numbered items, bold spans, code, emoji); **Vocabulary** (mean word length, type-token ratio of the first 100 words, share of long words, first-person pronouns); **Punctuation** (commas, exclamation marks, question marks, colons, semicolons, parentheses and dashes per 100 words); **Habits** (opens with *Certainly/Sure*, repeats the question, closes with an offer of help); and **Encoding** (number of U+FFFD replacement characters). Normalizing per 100 words and measuring lexical diversity on a fixed window prevents these features from simply re-encoding length.

A gradient-boosting classifier using only these 28 numbers reaches **0.546 macro-F1** (accuracy 0.552), and a linear logistic regression 0.429, versus 0.059 chance. So a handful of interpretable statistics already identifies the source in more than half of the cases, but the full-text models (0.72 to 0.77) still capture considerably more.

4.2 (i) Which signals does the classifier use?

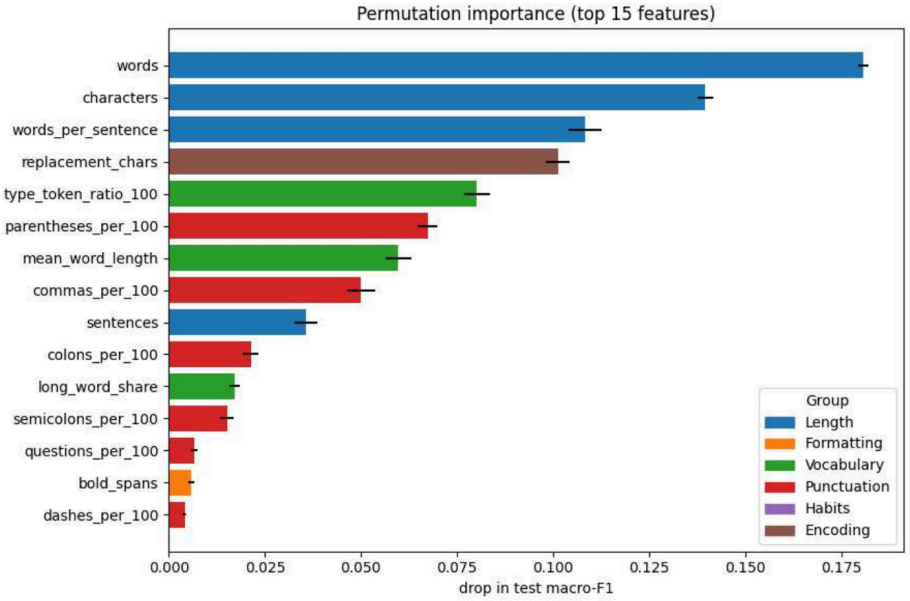


Figure 6. Permutation importance: drop in test macro-F1 when one feature is randomly shuffled (gradient boosting, 5 repeats).

Length dominates: shuffling the word count alone costs 0.18 macro-F1, followed by characters and words per sentence. Next come lexical diversity, mean word length and punctuation rates (parentheses, commas, colons, semicolons). The most indicative n-grams of the TF-IDF model point the same way: sources differ in their favorite connective phrases, for example *in summary* (BLACKBOXAI), *overall* (DeepAI), *additionally* (Nova, Poe, BLACKBOXAI), *ultimately* (Copilot), *for instance* and *for example* (Qwen) and *like* / *as seen in* (Grok).

A dataset artifact. The fourth most important feature is the number of U+FFFD replacement characters, and these characters are also among the top n-grams for ChatGPT, Copilot, DeepSeek, Gemma, Grok, Perplexity, Pi and Qwen. They mostly appear where an apostrophe or quotation mark should be (for example in possessive *'s*), so they are probably curly quotes that were damaged when the data was stored. This reflects either the LLMs' typography (curly versus straight quotes) or how each source's text was collected. We cannot separate the two, so we report it as an artifact rather than a genuine writing-style fingerprint.

4.3 (ii) Linguistic, structural, or both?

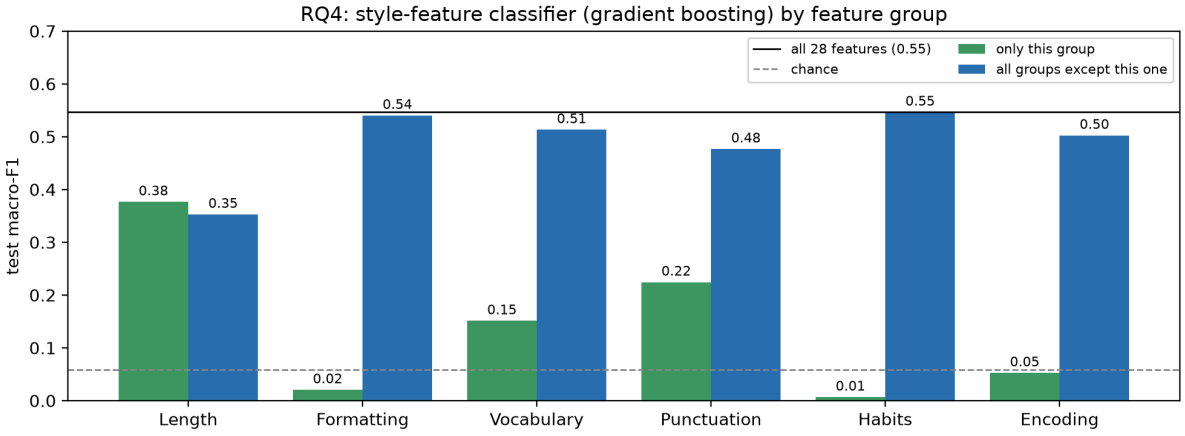


Figure 7. Style-feature classifier using only one feature group (green) or all groups except one (blue).

Both, but length and punctuation lead. Length alone reaches 0.376 macro-F1 and removing it causes the largest drop (to 0.353). Punctuation alone reaches 0.224 and vocabulary statistics 0.151. By contrast, **formatting alone is at chance (0.020)** and removing it changes almost nothing. During training, scikit-learn warned that some formatting features are constant (they never occur), and only bold spans (****word****) appear among the important features. This suggests the dataset stores answers largely without line breaks or Markdown structure, so the formatting fingerprint that chat interfaces show (headings, bullet lists) cannot be studied with this data. The hand-crafted habits (opening affirmations, closing offers) are also uninformative, possibly because the stored answers rarely include such phrases.

4.4 (iii) Does removing signals reduce performance?

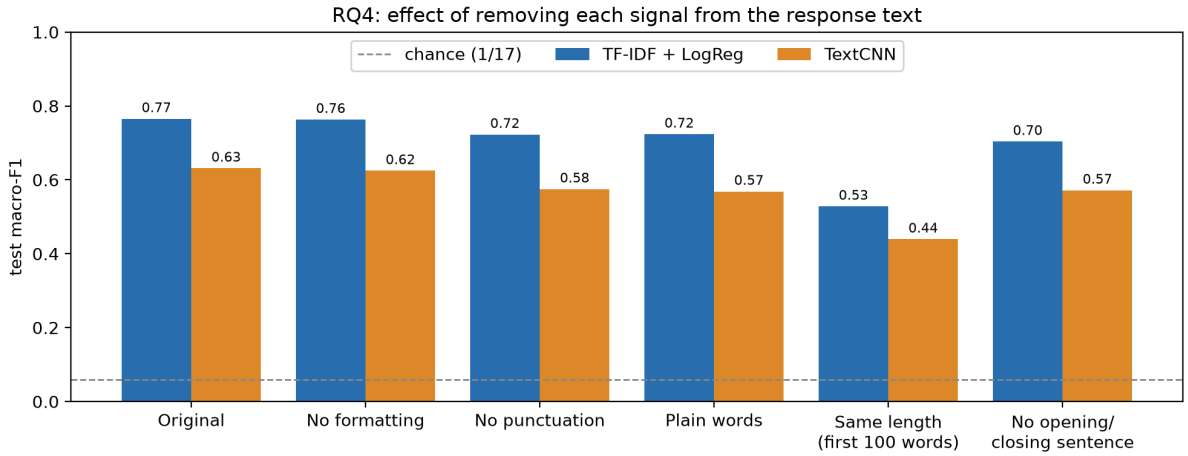


Figure 8. Removal experiments. The signal is removed from every train and test response, then the models are retrained.

Variant	TF-IDF macro-F1	Change	TextCNN macro-F1	Change
Original	0.765		0.632	
No formatting (Markdown symbols, line breaks)	0.763	-0.002	0.625	-0.007
No punctuation	0.723	-0.043	0.575	-0.056
Plain words (no formatting, punctuation or case)	0.724	-0.042	0.569	-0.063
Same length (first 100 words only)	0.528	-0.237	0.439	-0.192
No opening and closing sentence	0.704	-0.062	0.572	-0.060

Table 5. Removal experiments. The TF-IDF model here uses the punctuation-aware tokenizer, which is why its original score (0.765) is higher than in RQ1 (0.723).

- **Formatting:** removing it costs almost nothing, consistent with Section 4.3.
- **Punctuation:** removing it costs 0.04 to 0.06. Punctuation-aware TF-IDF is also 0.04 better than RQ1's word-only TF-IDF, so punctuation habits are a real signal.
- **Opening and closing sentences:** removing them costs about 0.06, showing that formulaic introductions and conclusions (*In summary, Overall,*) are distinctive.
- **Length:** truncating every answer to 100 words causes by far the largest drop (0.19 to 0.24). This removes the length signal but also about two thirds of the text, so part of the drop is simply less evidence. Even so, both models stay far above chance (0.44 to 0.53), so 100 words already contain a recognizable fingerprint.
- **Word choice survives everything:** with only lower-cased words left (*Plain words*), TF-IDF still reaches 0.724. The fingerprint is therefore mainly *lexical*: which words and phrases a model prefers, and how much it writes.

5. Lessons and Experience

- **Look at the labels before modeling.** Our first RQ1 run used all 22 sources. Only by reading the confusion matrix did we notice that five labels were human or mixed human/AI text, which do not belong in an LLM-identification task. Checking the label set first would have saved a full training run.
- **The split defines what the result means.** Splitting by question was essential to test generalization to new prompts. The RQ2 leakage check showed that a naive random split does not just inflate results; here it produced scores far below chance. Reasoning about how the data was generated is as important as the model.
- **Strong simple baselines are useful.** TF-IDF + logistic regression trained in about 90 seconds and beat both neural networks. Without it we might have believed 0.63 was near the ceiling. The networks overfit within 5 to 8 epochs, so early stopping on validation macro-F1 was necessary, and pre-trained embeddings or a pre-trained transformer would be the natural next step.
- **Datasets carry artifacts.** Broken encoding characters became one of the most important features, and formatting information appears to have been lost when the data was stored. A classifier will exploit any difference between classes, wanted or not, so feature-importance and removal analyses are needed before claiming that a model has learned an *LLM fingerprint*.

Limitations and future work. Each experiment was run once with one split, so we cannot give confidence intervals; repeating with several seeds and splits would. We did not tune hyperparameters, and RQ4's removal experiments use only the CNN to limit runtime. RQ3 (generalization across domains) was not attempted; it would require labeling the questions by domain first.

6. Reproducibility

All code is in our repository. Each research question is one self-contained notebook that runs top to bottom on a Colab GPU and writes its figures and tables to *results/*: *rq1_llm_attribution.ipynb*, *rq2_prompt_vs_response.ipynb* and *rq4_fingerprint_analysis.ipynb*. Libraries: PyTorch, scikit-learn, pandas, NumPy and matplotlib.

Use of AI tools. As permitted by the course policy, we used an AI assistant (Claude) to help write and debug code and to draft and edit this report. All experiments were run by us, and all numbers in this report come from our own runs.